

Designing with RTOS

Multiprocessing & Multitasking

- ✓ The ability to execute multiple processes simultaneously is referred as *multiprocessing*
- ✓ Systems which are capable of performing multiprocessing are known as *multiprocessor* systems
- ✓ *Multiprocessor* systems possess multiple CPUs and can execute multiple processes simultaneously
- ✓ The ability of the Operating System to have multiple programs in memory, which are ready for execution, is referred as *multiprogramming*
- ✓ *Multitasking* refers to the ability of an operating system to hold multiple processes in memory and switch the processor (CPU) from executing one process to another process

Designing with RTOS

Multiprocessing & Multitasking

- ✓ *Multitasking* involves ‘*Context switching*’, ‘*Context saving*’ and ‘*Context retrieval*’
- ✓ *Context switching* refers to the switching of execution context from task to other
- ✓ When a task/process switching happens, the current context of execution should be saved to (*Context saving*) retrieve it at a later point of time when the CPU executes the process, which is interrupted currently due to execution switching
- ✓ During context switching, the context of the task to be executed is retrieved from the saved context list. This is known as *Context retrieval*

Designing with RTOS

Preemptive scheduling

- ✓ Employed in systems, which implements preemptive multitasking model
- ✓ Every task in the '*Ready*' queue gets a chance to execute. When and how often each process gets a chance to execute (gets the CPU time) is dependent on the type of preemptive scheduling algorithm used for scheduling the processes
- ✓ The scheduler can preempt (stop temporarily) the currently executing task/process and select another task from the '*Ready*' queue for execution
- ✓ When to pre-empt a task and which task is to be picked up from the '*Ready*' queue for execution after preempting the current task is purely dependent on the scheduling algorithm
- ✓ A task which is preempted by the scheduler is moved to the '*Ready*' queue. The act of moving a '*Running*' process/task into the '*Ready*' queue by the scheduler, without the processes requesting for it is known as '*Preemption*'
- ✓ Time-based preemption and priority-based preemption are the two important approaches adopted in preemptive scheduling

Designing with RTOS

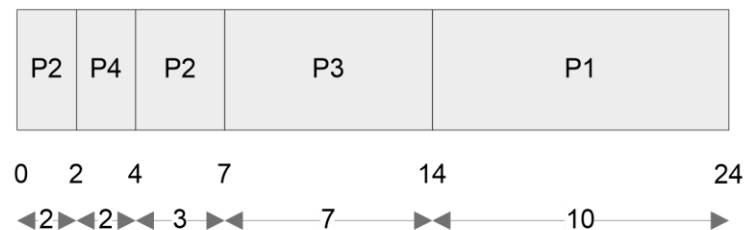
Preemptive scheduling – Preemptive SJF Scheduling/ Shortest Remaining Time (SRT)

- ✓ The *non preemptive SJF* scheduling algorithm sorts the ‘Ready’ queue only after the current process completes execution or enters wait state, whereas the *preemptive SJF* scheduling algorithm sorts the ‘Ready’ queue when a new process enters the ‘Ready’ queue and checks whether the execution time of the new process is shorter than the remaining of the total estimated execution time of the currently executing process
- ✓ If the execution time of the new process is less, the currently executing process is preempted and the new process is scheduled for execution
- ✓ Always compares the execution completion time (ie the remaining execution time for the new process) of a new process entered the ‘Ready’ queue with the remaining time for completion of the currently executing process and schedules the process with shortest remaining time for execution

Designing with RTOS

Preemptive scheduling – Preemptive SJF Scheduling

- Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds respectively enter the ready queue together. A new process P4 with estimated completion time 2ms enters the 'Ready' queue after 2ms. Assume all the processes contain only CPU operation and no I/O operations are involved.
- At the beginning, there are only three processes (P1, P2 and P3) available in the 'Ready' queue and the SRT scheduler picks up the process with the Shortest remaining time for execution completion (In this example P2 with remaining time 5ms) for scheduling. Now process P4 with estimated execution completion time 2ms enters the 'Ready' queue after 2ms of start of execution of P2. The processes are re-scheduled for execution in the following order



Designing with RTOS

Non-preemptive scheduling – Preemptive SJF Scheduling

The waiting time for all the processes are given as

Waiting Time for P2 = 0 ms + (4 - 2) ms = 2ms (P2 starts executing first and is interrupted by P4 and has to wait till the completion of P4 to get the next CPU slot)

Waiting Time for P4 = 0 ms (P4 starts executing by preempting P2 since the execution time for completion of P4 (2ms) is less than that of the Remaining time for execution completion of P2 (Here it is 3ms))

Waiting Time for P3 = 7 ms (P3 starts executing after completing P4 and P2)

Waiting Time for P1 = 14 ms (P1 starts executing after completing P4, P2 and P3)

Average waiting time = (Waiting time for all the processes) / No. of Processes
 = (Waiting time for (P4+P2+P3+P1)) / 4
 = (0 + 2 + 7 + 14)/4 = 23/4
 = 5.75 milliseconds

Turn Around Time (TAT) for P2 = 7 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 2 ms

(Time spent in Ready Queue + Execution Time = (Execution Start Time – Arrival Time) + Estimated Execution Time = (2-2) + 2)

Turn Around Time (TAT) for P3 = 14 ms (Time spent in Ready Queue + Execution Time)

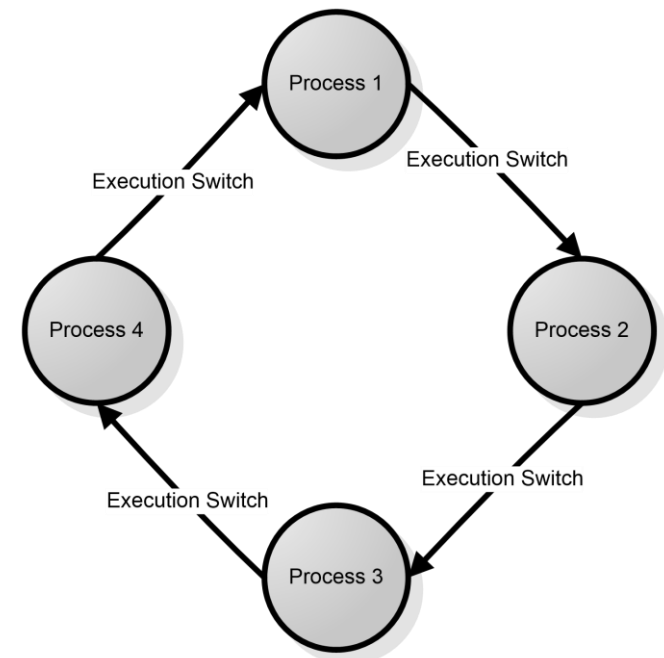
Turn Around Time (TAT) for P1 = 24 ms (Time spent in Ready Queue + Execution Time)

Average Turn Around Time = (Turn Around Time for all the processes) / No. of Processes
 = (Turn Around Time for (P2+P4+P3+P1)) / 4
 = (7+2+14+24)/4 = 47/4
 = 11.75 milliseconds

Designing with RTOS

Preemptive scheduling – Round Robin (RR) Scheduling

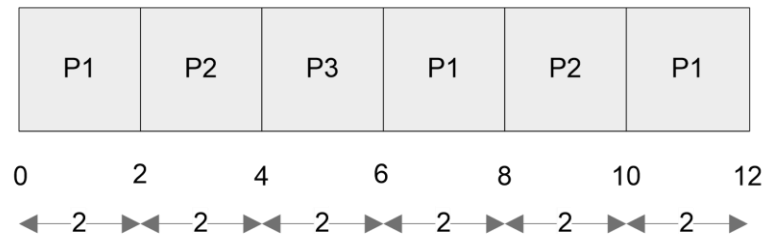
- ✓ Each process in the 'Ready' queue is executed for a pre-defined time slot.
- ✓ The execution starts with picking up the first process in the 'Ready' queue. It is executed for a pre-defined time
- ✓ When the pre-defined time elapses or the process completes (before the pre-defined time slice), the next process in the 'Ready' queue is selected for execution.
- ✓ This is repeated for all the processes in the 'Ready' queue
- ✓ Once each process in the 'Ready' queue is executed for the pre-defined time period, the scheduler comes back and picks the first process in the 'Ready' queue again for execution
- ✓ Round Robin scheduling is similar to the FCFS scheduling and the only difference is that a time slice based preemption is added to switch the execution between the processes in the 'Ready' queue



Designing with RTOS

Preemptive scheduling – Round Robin Scheduling

- Three processes with process IDs P1, P2, P3 with estimated completion time 6, 4, 2 milliseconds respectively, enters the ready queue together in the order P1, P2, P3. Calculate the waiting time and Turn Around Time (TAT) for each process and the Average waiting time and Turn Around Time (Assuming there is no I/O waiting for the processes) in RR algorithm with Time slice= 2ms.
- The scheduler sorts the 'Ready' queue based on the FCFS policy and picks up the first process P1 from the 'Ready' queue and executes it for the time slice 2ms. When the time slice is expired, P1 is preempted and P2 is scheduled for execution. The Time slice expires after 2ms of execution of P2. Now P2 is preempted and P3 is picked up for execution. P3 completes its execution within the time slice and the scheduler picks P1 again for execution for the next time slice. This procedure is repeated till all the processes are serviced. The order in which the processes are scheduled for execution is represented as



Designing with RTOS

Non-preemptive scheduling – Round Robin Scheduling

The waiting time for all the processes are given as

Waiting Time for P1 = $0 + (6-2) + (10-8) = 0+4+2 = 6\text{ms}$ (P1 starts executing first and waits for two time slices to get execution back and again 1 time slice for getting CPU time)

Waiting Time for P2 = $(2-0) + (8-4) = 2+4 = 6\text{ms}$ (P2 starts executing after P1 executes for 1 time slice and waits for two time slices to get the CPU time)

Waiting Time for P3 = $(4 - 0) = 4\text{ms}$ (P3 starts executing after completing the first time slices for P1 and P2 and completes its execution in a single time slice.)

Average waiting time = (Waiting time for all the processes) / No. of Processes
 = (Waiting time for (P1+P2+P3)) / 3
 = $(6+6+4)/3 = 16/3$
 = 5.33 milliseconds

Turn Around Time (TAT) for P1 = 12 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 10 ms (-Do-)

Turn Around Time (TAT) for P3 = 6 ms (-Do-)

Average Turn Around Time = (Turn Around Time for all the processes) / No. of Processes
 = (Turn Around Time for (P1+P2+P3)) / 3
 = $(12+10+6)/3 = 28/3$
 = 9.33 milliseconds

Designing with RTOS

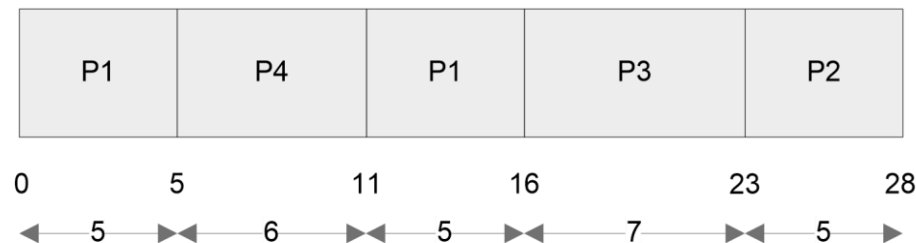
Preemptive scheduling – Priority based Scheduling

- ✓ Same as that of the *non-preemptive priority* based scheduling except for the switching of execution between tasks
- ✓ In *preemptive priority* based scheduling, any high priority process entering the 'Ready' queue is immediately scheduled for execution whereas in the *non-preemptive* scheduling any high priority process entering the 'Ready' queue is scheduled only after the currently executing process completes its execution or only when it voluntarily releases the CPU
- ✓ The priority of a task/process in preemptive priority based scheduling is indicated in the same way as that of the mechanisms adopted for non-preemptive multitasking

Designing with RTOS

Preemptive scheduling – Priority based Scheduling

- Three processes with process IDs P1, P2, P3 with estimated completion time 10, 5, 7 milliseconds and priorities 1, 3, 2 (0- highest priority, 3 lowest priority) respectively enters the ready queue together. A new process P4 with estimated completion time 6ms and priority 0 enters the 'Ready' queue after 5ms of start of execution of P1. Assume all the processes contain only CPU operation and no I/O operations are involved.
- At the beginning, there are only three processes (P1, P2 and P3) available in the 'Ready' queue and the scheduler picks up the process with the highest priority (In this example P1 with priority 1) for scheduling. Now process P4 with estimated execution completion time 6ms and priority 0 enters the 'Ready' queue after 5ms of start of execution of P1. The processes are re-scheduled for execution in the following order



Designing with RTOS

Preemptive scheduling – Priority based Scheduling

The waiting time for all the processes are given as

Waiting Time for P1 = $0 + (11-5) = 0+6 = 6$ ms (P1 starts executing first and gets preempted by P4 after 5ms and again gets the CPU time after completion of P4)

Waiting Time for P4 = 0 ms (P4 starts executing immediately on entering the 'Ready' queue, by preempting P1)

Waiting Time for P3 = 16 ms (P3 starts executing after completing P1 and P4)

Waiting Time for P2 = 23 ms (P2 starts executing after completing P1, P4 and P3)

$$\begin{aligned}\text{Average waiting time} &= (\text{Waiting time for all the processes}) / \text{No. of Processes} \\ &= (\text{Waiting time for (P1+P4+P3+P2)}) / 4 \\ &= (6 + 0 + 16 + 23)/4 = 45/4 \\ &= 11.25 \text{ milliseconds}\end{aligned}$$

Turn Around Time (TAT) for P1 = 16 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P4 = 6ms (Time spent in Ready Queue + Execution Time = (Execution Start Time – Arrival Time) + Estimated Execution Time = $(5-5) + 6 = 0 + 6$)

Turn Around Time (TAT) for P3 = 23 ms (Time spent in Ready Queue + Execution Time)

Turn Around Time (TAT) for P2 = 28 ms (Time spent in Ready Queue + Execution Time)

$$\begin{aligned}\text{Average Turn Around Time} &= (\text{Turn Around Time for all the processes}) / \text{No. of Processes} \\ &= (\text{Turn Around Time for (P2+P4+P3+P1)}) / 4 \\ &= (16+6+23+28)/4 = 73/4 \\ &= 18.25 \text{ milliseconds}\end{aligned}$$

Designing with RTOS

Preemptive scheduling – Priority based Scheduling

Summary:

- ✓ Priority based preemptive scheduling gives real time attention to high priority tasks
- ✓ Priority based preemptive scheduling is adopted in systems which demands ‘Real Time’ behavior
- ✓ Most of the RTOSs implements the preemptive priority based scheduling algorithm for process/task scheduling
- ✓ Preemptive priority based scheduling also possess the same drawback of non-preemptive priority based scheduling – ‘*Starvation*’
- ✓ This can be eliminated by the ‘*Aging*’ technique (Temporarily boosting the priority of a task which is ‘starving’ for a long time)

Multitasking and Task Scheduling

Learning Summary

- ✓ The ability of a system to execute multiple processes/tasks simultaneously is known as multiprocessing, whereas the ability of an operating system to hold multiple processes in memory and switch the processor (CPU) from executing one process to another process is known as multitasking.
- ✓ Multitasking involves *Context Switching*, *Context Saving* and *Context Retrieval*
- ✓ *Co-operative* multitasking, *Preemptive* multitasking and *Non-preemptive* multitasking are the three important types of multitasking which exist in the Operating system context
- ✓ CPU utilization, Throughput, Turn Around Time (TAT), Waiting Time and Response Time are the important criteria that need to be considered for the selection of a scheduling algorithm for task scheduling
- ✓ A good scheduling algorithm provides high CPU utilization, minimum Turn Around Time (TAT), maximum throughput and least response time.
- ✓ Job queue, Ready queue and Device queue are the important queues maintained by an operating system in association with CPU scheduling
- ✓ First Come First Served (FCFS)/First in First Out (FIFO), Last Come First Served (LCFS)/Last in First Out (LIFO), Shortest Job First (SJF), priority based scheduling etc are examples for Non-preemptive scheduling,
- ✓ Preemptive SJF Scheduling/ Shortest Remaining Time (SRT), Round Robin (RR) scheduling and priority based scheduling are examples for preemptive scheduling